LDAP partie 1 - Prise en main de OpenLDAP sous Debian

Recherche de la cible à installer

```
#apt search ldap
...
slapd ...
...

#apt show slapd
...
```

Les commentaires sur le paquet slapd confirment la cible.

Analyse du comportement de slapd

L'installation de base, **sur une machine propre** se déroule sans intervention utilisateur, hormis la saisie d'un mot de passe pour l'administrateur (ici, pxd : admin).

Le démon `slapd` semble être actif :

```
# ps auxf
...
openldap ... sr/sbin/slapd -d0 -h ldap:/// ldapi:/// -u openldap -g openldap
```

L'installation a donc généré une configuration valide, sauf que le DIT nous en est inconnu.

Recherche d'informations complémentaires

```
# dpkg -L slapd
...
/usr/share/doc/slapd/README.Debian.gz
...

# zcat /usr/share/doc/slapd/README.Debian.gz
...
You can use the following shell command, as root, to search the
  configuration:
      ldapsearch -Y EXTERNAL -H ldapi:/// -b "cn=config"
...
```

Détaillons le résultat de cette commande, en posant le paradigme que `cn=config` est la base (Root, ou sommet du DIT),sous forme d'un arbre

```
cn=config
    cn=module{0}
```

```
        ...
    cn=shema
        cn={0}core
        cn={1}cosine
        ...
    olcDataBase={-1}frontend
        ...
    olcDataBase{0}config
        ...
    oldDataBase{1}mdb
        dn: olcDatabase={1}mdb,cn=config
        objectClass: olcDatabaseConfig
        objectClass: olcMdbConfig
        olcDatabase: {1}mdb
        olcDbDirectory: /var/lib/ldap
        olcSuffix: dc=nodomain
        olcAccess: {0}to attrs=userPassword by self write by anonymous auth by * none
        olcAccess: {1}to attrs=shadowLastChange by self write by * read
        olcAccess: {2}to * by * read
        olcRootDN: cn=admin,dc=nodomain
        olcRootPW: {SSHA}koJHThqcHFULem5N+IkN6FWTAnPtPNy9
        olcDbIndex: objectClass eq
        olcDbIndex: cn,uid eq
        olcDbIndex: uidNumber,gidNumber eq
        olcDbIndex: member,memberUid eq
        olcDbMaxSize: 1073741824
```

Donc, par défaut, Debian contruit une BDD sous le contexte dc=nodomain. On peut constater l'absence d'ACL concernant `cn=admin,dc=nodomain`. Une petite recherche sur l'Internet nous di que c'est normal, au moins depuis OpenLDAP v2.5, suite à l'introduction de la configuration dynamique : ***OpenLDAP ne nécessite pas d'ACL pour le compte racine, car il est toujours autorisé***.

Mode *debug* et `journald`

Pour l'instant, les traces disponibles dans journald ne permettent pas un "débugage" avancé du fonctionnement.

Activation du débugage

L'usage est simple, créer un fichier `ldif` et l'appliquer.

```
# cat log_acl.ldif
    dn: cn=config
    changetype: modify
    replace: olcLogLevel
    olcLogLevel: acl
```

```
# ldapmodify -Y EXTERNAL -H ldapi:/// -f log_acl.ldif
    ...
    modifying entry "cn=config"
```

Utilisez `# journalctl -u slapd -f`dans une autre console pour voir les logs au fil de l'eau.

Retour au débugage de base

Le niveau de debugage de base est *none*. pour l'appliquer :

```
# cat log_none.ldif
    dn: cn=config
    changetype: modify
    replace: olcLogLevel
    olcLogLevel: none

# ldapmodify -Y EXTERNAL -H ldapi:/// -f log_none.ldif
    ...
    modifying entry "cn=config"
```

Personalisation du contexte

Retrait du suffixe `dc=nodomain`

L'usage est simple, là aussi. Créer un fichier `ldif` et l'appliquer.

Le fichier `ldif`contiendra :

```
# cat del_db.ldif
dn: olcDatabase={1}mdb,cn=config
changetype: delete
```

Il sera appliqué avec la commande :

```
# ldapmodify -Y EXTERNAL -H ldapi:/// -f del_db.ldif
SASL/EXTERNAL authentication started
SASL username: gidNumber=0+uidNumber=0,cn=peercred,cn=external,cn=auth
SASL SSF: 0
deleting entry "olcDatabase={1}mdb,cn=config"
```

Création du suffixe `dc=facerias, dc=ord`

L'usage est le même, créer un fichier `ldif` et l'appliquer.

Le fichier `ldif`contiendra :

```
# cat add_suffix.ldif
dn: olcDatabase=mdb,cn=config
objectClass: olcDatabaseConfig
```

```
objectClass: olcMdbConfig
olcDatabase: mdb
olcSuffix: dc=facerias,dc=org
olcRootDN: cn=admin,dc=facerias,dc=org
olcRootPW: {SSHA}koJHThqcHFULem5N+IkN6FWTAnPtPNy9
olcDbDirectory: /var/lib/ldap
olcDbIndex: objectClass eq
olcDbIndex: cn,uid eq
olcDbIndex: uidNumber,gidNumber eq
olcDbIndex: member,memberUid eq
olcDbMaxSize: 1073741824
olcAccess: {0}to attrs=userPassword by self write by anonymous auth by * none
olcAccess: {1}to attrs=shadowLastChange by self write by * read
olcAccess: {2}to * by * read
```

/! NOTA : le PW est `admin` !! Il sera appliqué avec la commande :

```
# ldapadd -Y EXTERNAL -H ldapi:/// -f add_suffix.ldif
SASL/EXTERNAL authentication started
SASL username: gidNumber=0+uidNumber=0,cn=peercred,cn=external,cn=auth
SASL SSF: 0
adding new entry "olcDatabase=mdb,cn=config"
```

Le résultat se confirme par la la commande suivante :

```
# ldapsearch -Y EXTERNAL -H ldapi:/// -b "cn=config"
...
# {1}mdb, config
dn: olcDatabase={1}mdb,cn=config
objectClass: olcDatabaseConfig
objectClass: olcMdbConfig
olcDatabase: {1}mdb
olcDbDirectory: /var/lib/ldap
olcSuffix: dc=facerias,dc=org
olcAccess: {0}to attrs=userPassword by self write by anonymous auth by * none
olcAccess: {1}to attrs=shadowLastChange by self write by * read
olcAccess: {2}to * by * read
olcRootDN: cn=admin,dc=facerias,dc=org
olcRootPW: {SSHA}koJHThqcHFULem5N+IkN6FWTAnPtPNy9
...
```

L'utilisation d'un outil graphique comme *jxplorer* confirme un accès utilisant les paramètres suivants :

Par contre, le parcours de l'arborescence en dessous de `dc=faceris,dc=org` échoue car il n'y a rien à rechercher. C'est confirmé par le `search result` de la recherche suivante:

```
# ldapsearch -Y EXTERNAL -H ldapi:/// -b "dc=facerias,dc=org"
```

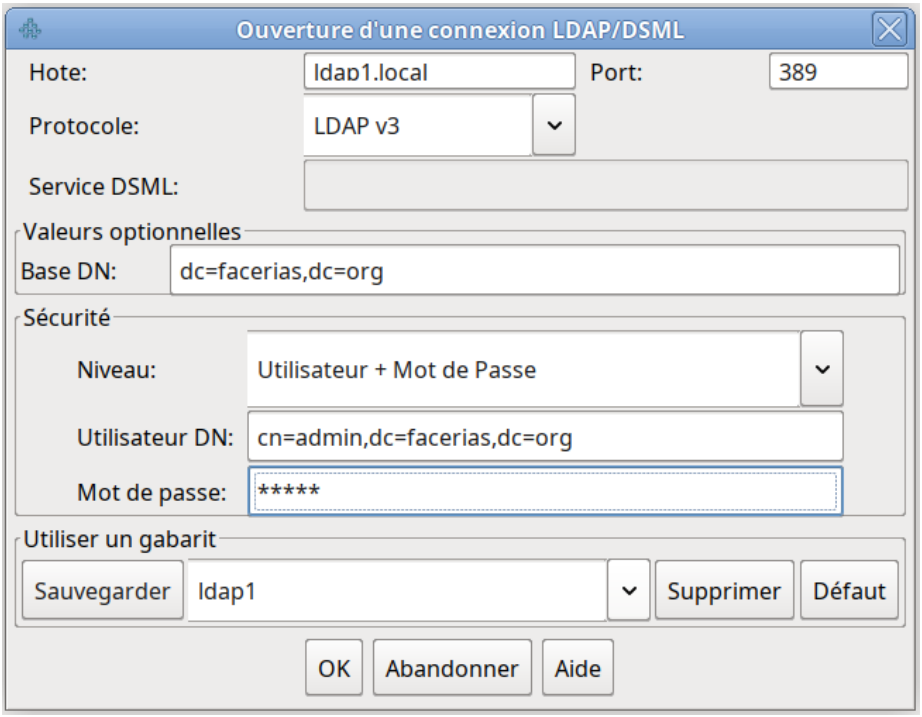

Figure 1: Accès via jxplorer

```
SASL/EXTERNAL authentication started
SASL username: gidNumber=0+uidNumber=0,cn=peercred,cn=external,cn=auth
SASL SSF: 0
# extended LDIF
#
# LDAPv3
# base <dc=facerias,dc=org> with scope subtree
# filter: (objectclass=*)
# requesting: ALL
#

# search result
search: 2
result: 32 No such object

# numResponses: 1
```

Peupler notre annuaire

Créer la BDD

La création du suffixe ne crée ni le contexte, ni la BDD elle-même (un suffixe peut *pointer* vers un autre serveur, dans le cas d'une cascade !)

L'usage est toujours le même, créer un fichier `ldif` et l'appliquer.

```
# cat mk_bdd.ldif
    dn: dc=facerias,dc=org
    objectClass: top
    objectClass: organization
    objectClass: dcObject
    o: Domaine facerias.org
    dc: facerias

# ldapadd -x -D "cn=admin,dc=facerias,dc=org" -w admin -f mk_bdd.ldif
```

Vous remarquerez que l'authentification se fait en utilisant le `olcRootDN`déclaré plus haut, l'hôte n'étant pas déclaré, c'est par défaut `localhost`.

Peupler la BDD

L'usage est encore le même, créer un fichier `ldif` et l'appliquer.

```
# cat fill_db.ldif
    # OU des personnes physiques
    dn: ou=people,dc=facerias,dc=org
    objectClass: organizationalUnit
    ou: people
```

```
# OU des rôles
dn: ou=roles,dc=facerias,dc=org
objectClass: organizationalUnit
ou: roles

# OU des groupes de users
dn: ou=groups,dc=facerias,dc=org
objectClass: organizationalUnit
ou: groups

# OU des groupes d'administration
dn: ou=admin,dc=facerias,dc=org
objectClass: organizationalUnit
ou: admin

# Création d'un user
dn: uid=mfacerias,ou=people,dc=facerias,dc=org
objectClass: inetOrgPerson
uid: mfacerias
sn: FACERIAS
givenName: Michel
cn: Michel FACERIAS
displayName: FACERIAS, Michel
userPassword: TmpPass
mail: michel@facerias.org
title: M.
initials: MFA


# Création du groupe adminldap
dn: cn=adminldap,ou=admin,dc=facerias,dc=org
objectClass: groupOfNames
cn: adminldap
member: uid=mfacerias,ou=people,dc=facerias,dc=org
```

```
# ldapadd -x -D "cn=admin,dc=facerias,dc=org" -w admin -f fill_bdd.ldif
```

Là encore, on utilise le `olcRootDN` de cette entrée.

Déléguer les droits au groupe `adminldap`

Pour l'instant, nous avons créé ce groupe, mais sans lui conférer de droits particuliers. Seul `mfacerias`en fait partie, mais que peut-il réellement faire ?

Essai de rajout d'une nouvelle entrée

Nous allons rajouter une entrée `test`, classiquement, mais en se limitant aux attributs obligatoires :

```
# cat add_test.ldif
    dn: uid=test,ou=people,dc=facerias,dc=org
    objectClass: inetOrgPerson
    uid: test
    cn: test
    sn: test

# ldapadd -x -D "uid=mfacerias,ou=people,dc=facerias,dc=org" -w pass -f add_test.ldif
adding new entry "uid=test,ou=people,dc=facerias,dc=org"
ldap_add: Insufficient access (50)
    additional info: no write access to parent
```

On pouvait s'y attendre, mais là, c'est confirmé, `mfacerias` n'a pas le droit.

Modification des privilèges du groupe `adminldap`

On va faire ça classiquement :

```
# cat acl_adminldap.ldif
dn: olcDatabase={1}mdb,cn=config
changetype: modify
add: olcAccess
olcAccess: {2}to * by group.exact="cn=adminldap,ou=admin,dc=facerias,dc=org" manage

# ldapadd -x   -D "cn=admin,dc=facerias,dc=org"   -w admin -f acl_adminldap.ldif
...
modifying entry "olcDatabase={1}mdb,cn=config"

# ldapadd -x -D "uid=mfacerias,ou=people,dc=facerias,dc=org" -w admin -f add_test.ldif
adding new entry "uid=test,ou=people,dc=facerias,dc=org"
```

L'entrée `test`a bien été rajoutée, elle est visible via *jxplorer*, par exemple.

Déléguer les droits à l'utilisateur système `root`

Pour l'instant, l'utilisateur système `root` n'a aucun droit sur le suffixe `dc=facerias,dc=org`. C'est génant à double titre : - on ne peut pas utiliser les droits de cet utilisateur pour la maintenance dans ce suffixe ; - on est obligé de maintenir `olcRootDN: cn=admin,dc=facerias,dc=org`pour cet usage. Même si nous avons confié = `mfacerias`des droits à travers le groupe `adminldap` il serait bon d'avoir la possibilité de maintenir la BDD de ce suffixe depuis le compte système `root`.

Essai de rajout d'une nouvelle entrée

Nous allons rajouter une entrée `test_byroot`, classiquement, mais en se limitant aux attributs obligatoires :

```
# cat add_test_byroot.ldif
    dn: uid=test_byroot,ou=people,dc=facerias,dc=org
    objectClass: inetOrgPerson
    uid: test_byroot
    cn: test_byroot
    sn: test_byroot

# ldapadd -Y EXTERNAL -H ldapi:/// -f add_test_byroot.ldif
    ...
    adding new entry "uid=test_byroot,ou=people,dc=facerias,dc=org"
    ldap_add: Insufficient access (50)
        additional info: no write access to parent
```

On s'y attendait, et c'est confirmé, `root` n'a pas le droit.

Modification des privilèges de l'utilisateur système `root`

On va faire ça classiquement :

```
# cat acl_bulk.ldif
    dn: olcDatabase={1}mdb,cn=config
    changetype: modify
    delete: olcAccess
    olcAccess: {2}
    -
    add: olcAccess
    olcAccess: {2}to * by group.exact="cn=adminldap,ou=admin,dc=facerias,dc=org" manage by c

# ldapadd -Y EXTERNAL -H ldapi:/// -f bulk.ldif
...
modifying entry "olcDatabase={1}mdb,cn=config"

# ldapadd -Y EXTERNAL -H ldapi:/// -f add_test_byroot.ldif
...
adding new entry "uid=test_byroot,ou=people,dc=facerias,dc=org"
```

L'entrée `test_byroot`a bien été rajoutée, elle est visible via *jxplorer*, par exemple.